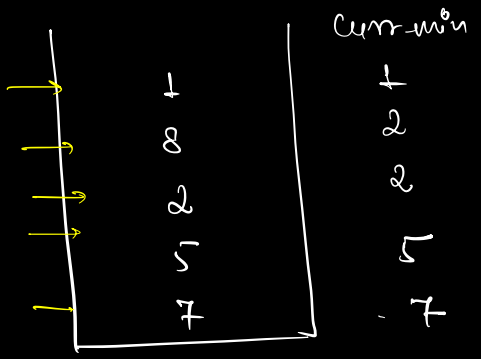
→ Min-Stack in O(1) space :-



push ( ) pop ( ) top ( ) getMin( )

↓ actual data from

↓ min at that pt.

$x < curr\_min$

$\Rightarrow x - curr\_min < 0$

add, x on both sides

$$2x - curr\_min < x$$

⇒ 1	2×1-2 = 0	curr_min = 1
⇒ 8	8	curr_min = 2
⇒ 2	2×2-5 = -1	curr_min = 2
⇒ 5	2×5-7 = 3	curr_min = 5
⇒ 7	7	curr_min = 7

```
push(x) {
    if( x <= curr_min ){
        st.push( 2x - curr_min)
        curr_min = x
    } else {
        st.push(x)
    }
}
```

```
top() {
    if( st.top() < curr_min)
        return curr_min
    else
        st.top()
}
```

```
pop() {
    if( st.top() < curr_min){
        original = curr_min
        prevCurrMin = 2x curr_min - st.top();
        curr_min = prevCurrMin
        st.pop();
        return original;
    }
    else
        return st.pop()
}
```

$2x - curr\_min < x$

→	1	2×1-2 = 0	curr_min = 1
	8		curr_min = 2
	2	2×2-5 = -1	curr_min = 2
→	5	2×5-7 = 3	curr_min = 5
	7		curr_min = 7

2 × 5 − minm before (7) = 3.
Pushing of 5

$2 \times 1 - 2 = 0$ ← st.top

$2 \times$ curr_min $-$ prevCurr_min $=$ st.top()

$\Rightarrow$ prevCurrMin $= 2 \times$ Curr_min $-$ st.top()

→ after popping

curr_Min = prevCurrMin

push(7)
push(5)
push(2)
getMin() → 2
push(8)
push(1)
getMin() → 1
top() → 1
pop() → 1
getMin() → 2

pop() → 8 actual data
top() → 2
getMin() → 2 ⇒ 1
⇒ 8
⇒ 2
⇒ 5
⇒ 7

	Curr min	prev Min
~~0~~	~~1~~	~~2~~
~~8~~	~~2~~	~~5~~
-1	2	5
3	5	7
7	7	X

curr_min : ~~7~~ ~~5~~ ~~2~~ ~~1~~ 2

Q1. Nearest Smaller element :- by position

For every index i, find the nearest smaller element on left side in comparison to arr[i].

	0	1	2	3	4	5
arr =	4	5	2	10	3	2
ans →	X	4	X	2	2	X
	-1		-1			-1

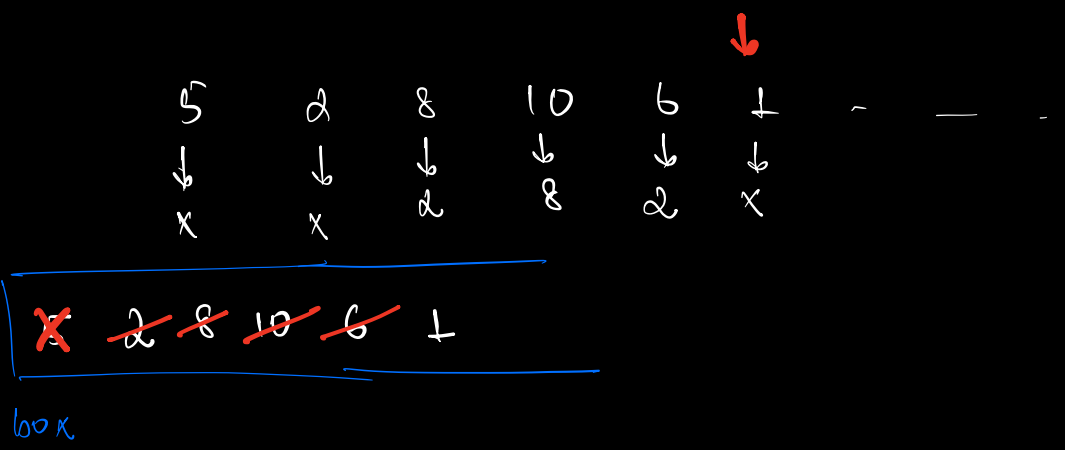
	0	1	2	3	4	5	6	7
	4	6	10	11	7	8	3	5
ans →	X	4	6	10	6	7	X	3
	-1						-1	

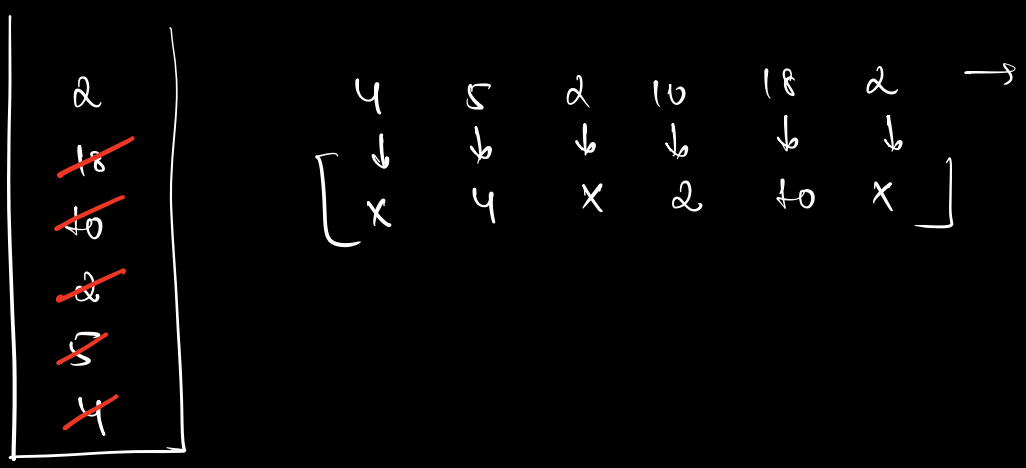
Basic :

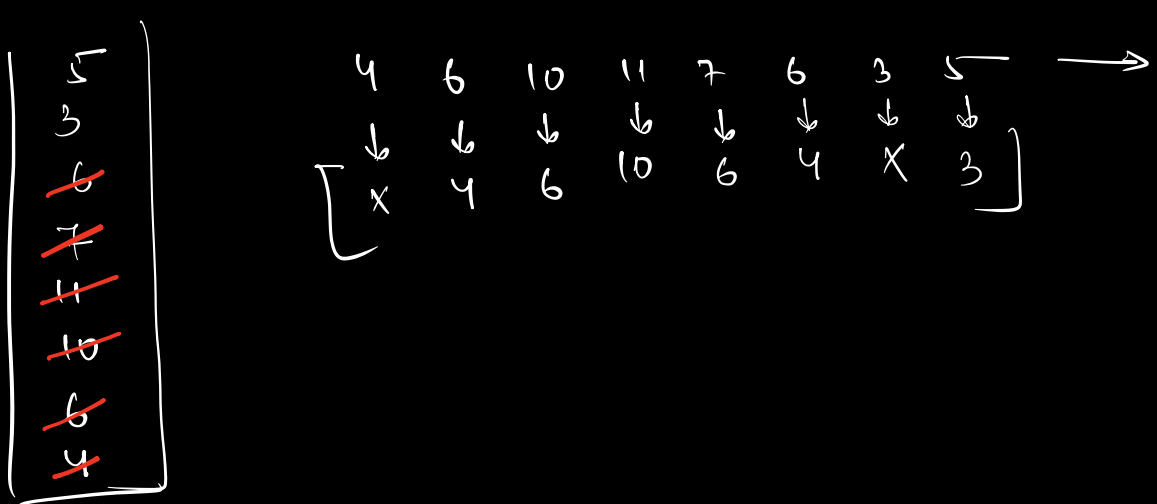
```
for(i=0 → N){
    ans[i] = -1
    for(j=i-1 → 0){
        if(arr[j] < arr[i]){
            ans[i] = arr[j]
            break
        }
    }
}
```

TC = O(N²)

SC = O(1) → no auxiliary space


5 2 8 10 6 1
x x 2 8 2 x
x 2 8 10 6 1
600 x


2
18
10
2
5
4
4 5 2 10 18 2
x 4 x 2 10 x


5
3
6
7
11
10
6
4
4 6 10 11 7 6 3 5
x 4 6 10 6 4 x 3

```
ans[], arr[]
for(i:0 ⟶ N){
while( !st.empty && st.top() >= arr[i] ){
        st.pop();
    }
    if( st.empty() ){
        ans[i] = -1
    } else {
        ans[i] = st.top();
    }
    st.push([arr[i]
}
```

→ change to <= for Q3

TC => O(N)
SC => O(N)

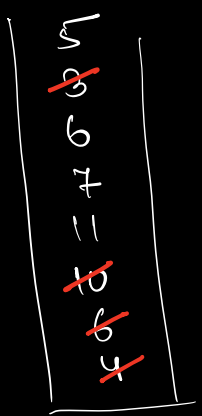
Q2. Nearest smaller element on right side ?

↓

iterate from N-1 → 0

Q 3. Nearest greater element — on left side

4	6	10	11	7	6	3	5
↓	↓	↓	↓	↓	↓	↓	↓
x	x	x	x	11	7	6	6

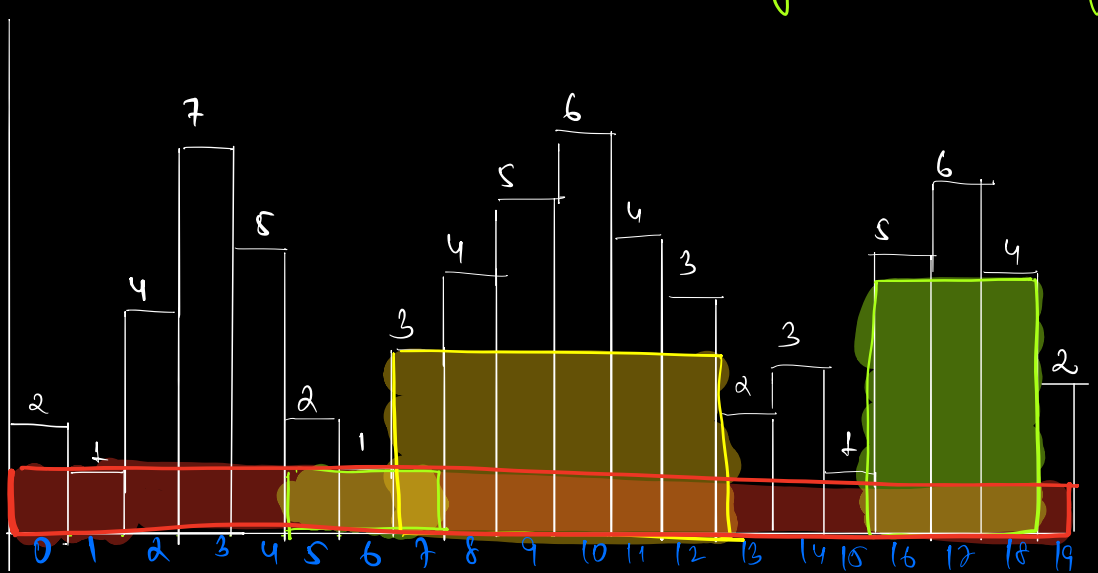


only change → while loop (<=)

Q 4. Nearest greater element — on right side.

→ Start from end in Q3

Q 5. Find the largest rectangle formed by continuous histogram



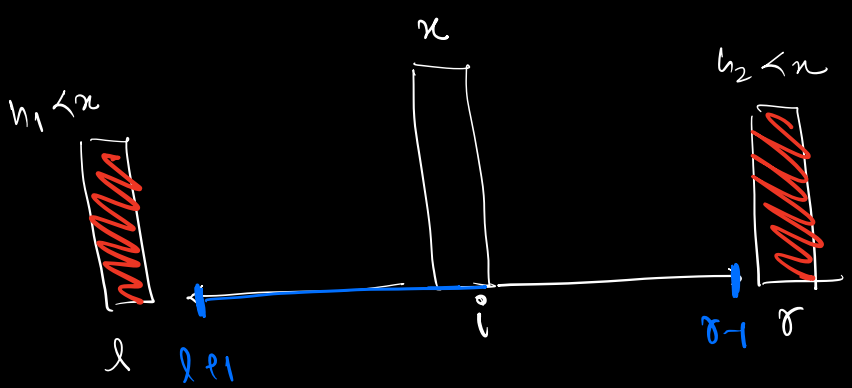
↓	↓	↓	↓
20	3	18	12

O/P ⇒ 20

B.F ⇒ consider all possible subarrays

↓

TC ⇒ $O(N^3)$ → optimised → $O(N^2)$



Area = height × width

= x × (r − l − 1)

(r−1) − (l+1) + 1

= r − 1 − l − 1 + 1

= r − l − 1

Calculate

ls[] ⇐ nearest smaller on left side (index)

rs[] ⇐ nearest smaller (index) on right side

for (i=0 → N) {

ans = max(ans, arr[i] × rs[i] − ls[i] − 1)

}

TC = O(N) ⟶ 3N → ls, rs, ans.

SC = O(N) ⟶ 2N → ls, rs.

Q 6. Find the sum of (max − min) for all subarrays.

subsequences

arr =

0	1	2
2	5	3

	max	min	max − min
0-0	2	2	0
0-1	5	2	3
0-2	5	2	3
1-1	5	5	0
1-2	5	3	2
2-2	3	3	0
			8 ⟶ Ans

BF ⇒ consider all subarrays → TC O(N²)

S_1 S_2 S_3 S_4

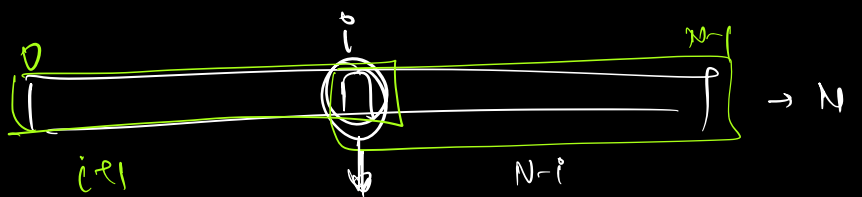
$(Max_1 - Min_1) + (Max_2 - Min_2) + (Max_3 - Min_3) + (Max_4 - Min_4)$

$= \left(\sum_{\text{Max of all subarrays}} - \sum_{\text{Min of all subarrays}} \right)$

↓

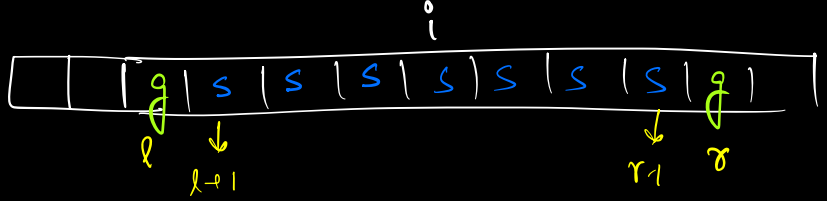
i is max in x

subarrays → contribution of i → arr[i] × x



subarrays where i is present = $(i-l) \times (r-i)$

Contribution technique → [find out — in how many subarrays i is max and i is min]



max =

$$arr[i] \times \left[\left(i - (lg+1) + 1 \right) \times \left(rg - i + 1 \right) \right]$$

lg rg

min =

$$arr[i] \times \left[\left(i - (ls+1) + 1 \right) \times \left(rs - i + 1 \right) \right]$$

ls rs

rs → smaller element on right side

ls → smaller element on left side

rg → greater element on right side

lg → greater element on left side

H.W → code